

ME 780 Computational Intelligence Final Report

**Application of Deep Deterministic
Policy Gradient (DDPG) and Decision
Transformer Reinforcement Learning to
Vehicle Path Following**

UNIVERSITY OF
WATERLOO



Fall 2024

Instructor: Dr. William Melek

Students: Ryan Tymkow & Benjamin Schnapp

Executive Summary

Path following is a task in which autonomous vehicles must complete to navigate through their environment safely. Current solutions to path following include classical or modern control systems like Proportional Integral Derivative (PID), Linear Quadratic Regulator (LQR), or Model Predictive Control (MPC) controllers, as well as learning-based methods such as Reinforcement Learning controllers, or end to end neural network systems.

The objective of this work is to develop and evaluate two forms of learning-based controllers, Deep Deterministic Policy Gradient (DDPG) Reinforcement Learning, and Decision Transformer Reinforcement Learning. In order to achieve this, a dynamics-based vehicle actor and a simulation environment were developed. The DDPG algorithm from Stable Baselines 3 was integrated with an OpenAI Gymnasium environment and trained to follow a path. Once the DDPG model was trained, it was deployed in the same environment and used to record data. The Hugging Face library was used to implement the Decision Transformer model, and the data recorded from the deployment of the DDPG model was used to train the Decision Transformer model.

Cross-track error, the distance from the vehicle's center of gravity to the closest point on the path, was used as the metric for evaluating the performance of each model. The DDPG model was able to quickly learn to follow the path and track the reference velocity, but did not perform well in sharp turns with the initial reward function implemented. In order to correct this, the reward function was modified so that the speed error was reduced by the curvature of the road. This resulted in the model learning to slow down in sharp turns, which decreased the average cross-track error. The average cross-track error from the final model was 0.505 meters with a standard deviation of 0.107 meters.

The decision transformer was extremely sensitive to hyperparameter tuning. With 800 episodes of data captured from the DDPG model, it was found that the decision transformer was only able to follow the path when using 2 epochs, a learning rate of 1×10^{-4} , and a batch size of 8. Another issue encountered with the decision transformer was the lack of diversity in the training dataset. The decision transformer was originally trained on good data from the DDPG model, and when the decision transformer model encountered a state out of the range of error seen in the training data, it would diverge from the path. To resolve this, the DDPG model was deployed into an environment with step discontinuities in the path which resulted in higher error in the training data. When the decision transformer was re-trained with the new data, it did not diverge from the path, but instead oscillated around the path, which resulted in high cross-track error. The average cross-track error from the decision transformer was 1.717 meters with a standard deviation of 0.372 meters.

A PID controller was also implemented to compare the results of learning-based driving methods with classical controllers. The PID controller had an average cross-track error of 0.784 meters, and a standard deviation of 0.247 meters. This shows that learning based methods have potential to outperform classical controllers, since the DDPG model had lower error than the PID controller.

Overall, the DDPG Reinforcement Learning model performed the best, followed by the PID controller, with the Decision Transformer performing the worst.

Table of Contents

1. Introduction	1
2. Methodology	1
2.1. Vehicle Model	1
2.2. Environment	4
2.2.1. Path Generation	5
2.2.2. Reward Function	6
2.3. Deep Deterministic Policy Gradient Algorithm	7
2.3.1. Implementation	7
2.3.2. Hyperparameter Tuning	7
2.4. Decision Transformer Algorithm	8
2.4.1. Implementation	10
2.4.2. Initial Hyperparameter Trials	10
2.4.3. Hyperparameter Tuning and Increasing Train Dataset Variance	11
3. Results	12
4. Discussion	14
5. Conclusion	15
5.1. Future Work	15
6. References	16
Appendix A: Deep Deterministic Policy Gradient Algorithm	21

1. Introduction

Path following is a critical task which autonomous vehicles, such as the WATonoBus developed by the University of Waterloo Mechatronics Vehicle Systems lab [1], must be able to complete to safely navigate in their environment. In a typical autonomous vehicle process flow, a vehicle must first complete perception of its environment, followed by path planning to decide on where to go, and lastly a control phase to enable the vehicle to execute actions and follow the decided path [2]. Path following and the work completed relates to the control phase of an autonomous driving process, where a vehicle must be able to follow a desired path which has already been determined by a path planning algorithm.

Currently, path following is largely achieved through conventional control methods, such as PID control [3], LQR control [4], or MPC control [5]. Reinforcement learning algorithms for vehicle path following have also been studied in recent papers, such as Cao et al. [6] or Hess et al.'s [7] implementations of vehicle path following using Deep Deterministic Policy Gradient (DDPG) reinforcement learning algorithms [8]. Reinforcement-learning-based approaches have shown similar or superior performance to conventional control methods, while reducing the work required for implementation and tuning compared to classical methods.

Two different reinforcement learning models were studied in this work. Deep deterministic policy gradient reinforcement learning models act as Q-Learning based reinforcement learning for continuous action spaces [8], [9]. Decision transformer reinforcement learning models are a more recently proposed method of reinforcement learning which function by using transformers and sequence modelling to select action maximizing long term rewards [10].

The objective of this work was to implement reinforcement-learning based control algorithms for path following and compare their performance to conventional control algorithms. Deep deterministic policy gradient [8] and decision transformer [10] reinforcement learning algorithms were selected and studied to determine their performance potential in path following applications.

2. Methodology

2.1. Vehicle Model

A dynamics-based model of the vehicle needed to be created in order to train a reinforcement learning model, and to test the results of any models or controllers developed. The Vehicle Bicycle Model [11] and Pacejka Magic Tire Model [12] were chosen as a simplified model for the complex vehicle dynamics. Additionally, drag force was added to the model to provide a realistic representation of longitudinal resistance, ensuring the

vehicle's behavior is more aligned with real-world dynamics and to prevent unbounded acceleration during simulation scenarios. The diagram of the Vehicle Bicycle Model is shown below in Figure 1, and Table 1 explains the common nomenclature used in the model.

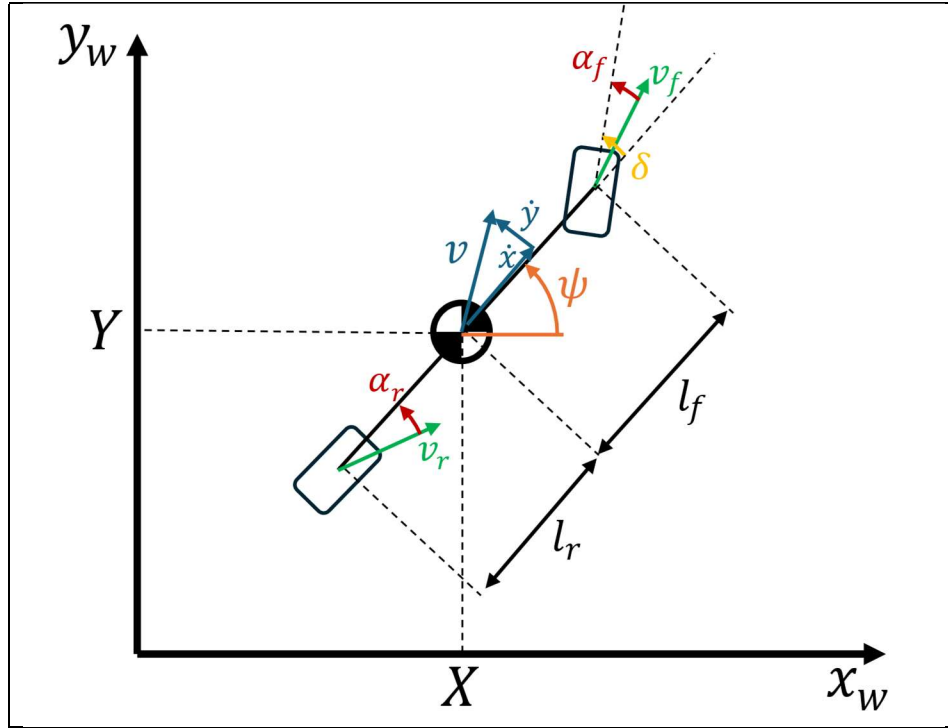


Figure 1: Vehicle Bicycle Model. Figure based on figure by Varma et AL. [11]

Table 1: Vehicle Model Nomenclature

X, Y	Vehicle center of mass position (global coordinate frame)
$\dot{x}, \dot{y}$	Velocity components (in vehicle coordinate frame)
x_w, y_w	World coordinate system
α_f, α_r	Front and rear tire slip angles
v_f, v_r	Front and rear axle velocity vectors
v	Velocity of vehicle center of mass
δ	Steering angle
l_f, l_r	Front and rear distance from axle to center of mass
ψ	Vehicle yaw angle (in global coordinate frame)
$\dot{\psi}$	Vehicle yaw rate
$F_{xr}, F_{yr}, F_{xf}, F_{yf}$	Forces in the x/y direction on the front/rear tires
I_z	Vehicle yaw moment of inertia
$C_{\alpha_f}, C_{\alpha_r}$	Front and rear tire cornering stiffness

The system dynamics in the vehicle base coordinate frame are given in equations (1), (2), and (3) below.

$$\ddot{x} m = F_{xr} + F_{xf} \cos \delta - F_{yf} \sin \delta - F_{drag} \quad (1)$$

Longitudinal Dynamics

$$\dot{y} m = F_{yr} + F_{xf} \sin \delta - F_{yf} \cos \delta \quad (2)$$

Lateral Dynamics

$$\ddot{\psi} I_z = l_f F_{xf} \sin \delta + l_r F_{yf} \cos \delta - l_r F_{yr} \quad (3)$$

Yaw Moment Dynamics

The cornering stiffness of the front and rear tires, $C_{\alpha f}$ and $C_{\alpha r}$, are used for a simple linear approximation of the Pacejka tire model. For small slip angles α , $F_{yf} \approx C_{\alpha f} \alpha_f$ and $F_{yr} \approx C_{\alpha r} \alpha_r$ [12]. The slip angles and corresponding tire forces are calculated below.

$$\alpha_f \approx \delta - \frac{\dot{y} + l_f \dot{\psi}}{\dot{x}} \rightarrow F_{yf} \approx C_{\alpha f} \left(\delta - \frac{\dot{y} + l_f \dot{\psi}}{\dot{x}} \right) \quad (4)$$

Front Tire Y Force from Pacejka Tire Model Approximation

$$\alpha_r \approx - \frac{\dot{y} - l_r \dot{\psi}}{\dot{x}} \rightarrow F_{yr} \approx C_{\alpha r} \left(- \frac{\dot{y} - l_r \dot{\psi}}{\dot{x}} \right) \quad (5)$$

Rear Tire Y Force from Pacejka Tire Model Approximation

Equations 4 and 5 were substituted into equations 1, 2 and 3 to obtain the full vehicle dynamics shown in equations 6, 7, and 8 below.

$$\ddot{x} = - \frac{C_d \rho A \dot{x}}{2m} + \frac{C_{\alpha f} \sin \delta}{m \dot{x}} \dot{y} + \frac{C_{\alpha f} \sin \delta l_f}{m \dot{x}} \dot{\psi} + \frac{1 + \cos \delta}{m} F_x - \frac{C_{\alpha f} \sin \delta}{m} \delta \quad (6)$$

Full Longitudinal Vehicle Dynamics

$$\ddot{y} = - \frac{C_{\alpha r} + C_{\alpha f} \cos \delta}{m \dot{x}} \dot{y} + \frac{C_{\alpha r} l_r - C_{\alpha f} l_f \cos \delta}{m \dot{x}} \dot{\psi} + \frac{\sin \delta}{2m} F_x - \frac{C_{\alpha f} \cos \delta}{m} \delta \quad (7)$$

Full Lateral Vehicle Dynamics

$$\ddot{\psi} = \frac{l_r C_{\alpha r} - l_f C_{\alpha f} \cos \delta}{I_z \dot{x}} \dot{y} - \frac{l_f^2 C_{\alpha f} \cos \delta + l_r^2 C_{\alpha r}}{I_z \dot{x}} \dot{\psi} + \frac{l_f \sin \delta}{2 I_z} F_x + \frac{l_f C_{\alpha f} \cos \delta}{I_z} \delta \quad (8)$$

Full Yaw Moment Vehicle Dynamics

Equations 6, 7, and 8 were converted into state space form. The states were $[\dot{x} \ \dot{y} \ \dot{\psi}]$, and the control inputs were $[F_x \ \delta]$. Numeric vehicle parameters, taken from the Tesla Model S, were then substituted into the model. The full list of vehicle parameters used are shown below in Table 2.

Table 2: Vehicle Parameters used for Model Simulation

m	1500 kg	I_f	1.2 m
I_z	3000 kg m ²	I_r	1.6 m
C_f	80000 N/rad	Cd	0.208
C_r	80000 N/rad	A	2.4 m ²
dt	0.1 s	ρ	1.225 kg / m ³

The vehicle dynamics state space model was then discretized with a discretization time step of 0.1 seconds using the zero order hold method from the Python Controls Library [13].

2.2. Environment

The Farama Foundation Gymnasium library [14] was used to simulate the vehicle model in an environment for the reinforcement learning agent to interact with. Gymnasium was selected as it provides a standardized API for implementing environments which are supported by a wide range of reinforcement learning algorithm libraries.

At each step the simulation was run, this discrete time vehicle state space equation was re-calculated using the previous state and parameters. The system was then solved, returning the states for the current time step.

A discrete time integrator was used to find the vehicles pose in the global coordinate frame, given by equations 9, 10, and 11 below.

$$\psi_t = \psi_{t-1} + \dot{\psi}_t dt \quad (9)$$

$$X_t = X_{t-1} + \dot{x}_t \cos(\psi_t) dt - \dot{y}_t \sin(\psi_t) dt \quad (10)$$

$$Y_t = Y_{t-1} + \dot{y}_t \cos(\psi_t) dt + \dot{x}_t \sin(\psi_t) dt \quad (11)$$

PyGame [15] was used as a visualization tool that would render and display the environment and vehicle model in real time.

2.2.1. Path Generation

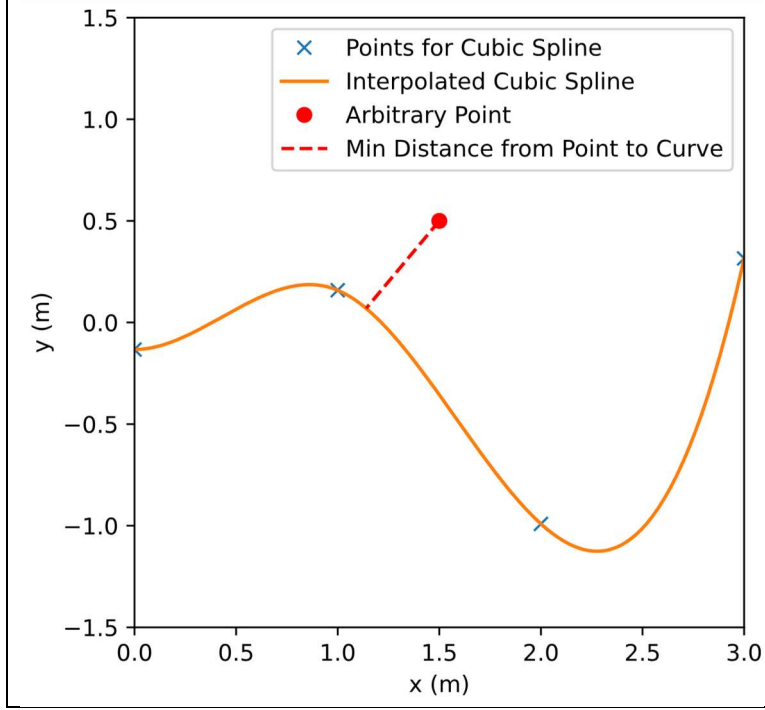


Figure 2: A sample curve generated by the path generation algorithm and minimum distance to an arbitrary point shown

The objective of this work was to develop a path following algorithm, so an algorithm to develop sample paths to follow was needed. Since path generation was not a priority of this work, a simple cubic spline interpolator was used. SciPy [16] was used for cubic spline interpolation in Python.

At $x = 0\text{m}$, a point was placed at $y = 0\text{m}$. At $x = 100\text{m}$, 200m , 300m , and 400m , the corresponding y coordinates were configured to an evenly distributed random number with $y \in (-100\text{m}, 100\text{m})$. A cubic spline was used to connect the points.

For the reward function, it was required to know the minimum distance from an arbitrary point (x_c, y_p) in 2D space to the curve. This was solved numerically. The cubic spline curve was discretized across $x_c \in [0\text{m}, 400\text{m}]$ with a discretization step of 0.1m , and the corresponding y_c points on the curve were found. An array storing the distance from the point to each point on the curve was solved simply using

$$d = \sqrt{(x_p - x_c)^2 + (y_p - y_c)^2} \quad (12)$$

Then the distance was taken as the point on the curve with the same index as the index of the minimum value of the distances array.

Although not a computationally efficient way to find the minimum distance, this calculation was insignificant compared to the computation time of updating and executing the reinforcement learning algorithms, and therefore had a negligible effect on the performance of running the simulated model for training.

The slope of the curve at the nearest point was also required for the heading error in the reward function. The slope was calculated using first-order finite differences between the nearest point and the discretized point one index after.

2.2.2. Reward Function

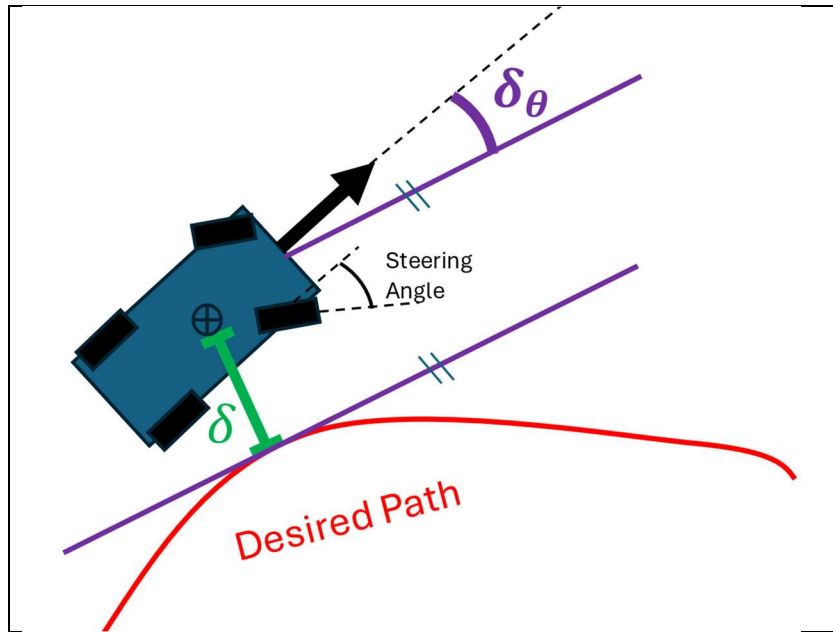


Figure 3: Vehicle State Parameters used for Reward Function

Initially, a reward function was defined purely based on the objective of minimizing the cross-track error δ

$$R = e^{-\delta} \quad (13)$$

The exponential form of the reward function was selected as it is differentiable, it produces a maximum output of 1 when the vehicle is directly on the path and exponentially decays to zero as the vehicle strays from the desired path.

It was found that with this simple initial tested reward function, the reinforcement learning algorithms trialed were able to converge to the path, but when faced with disturbances, oscillations arose, and the vehicle struggled to stably return to the path.

To address this, a modified reward function was developed which accounted for both the heading error δ_θ as well as the cross-track error δ .

$$R = e^{-\delta} * e^{-\delta_\theta} \quad (14)$$

This reward function was found to give good performance in training the models. Both summation and multiplication of the two rewards were attempted, and it was found that multiplication produced the best results, as it forced the agents to ensure that both cross-track and heading errors were kept low. With the summation reward function, on occasion, agents would converge to suboptimal policies where only one of the reward features was kept low.

2.3. Deep Deterministic Policy Gradient Algorithm

The deep deterministic policy gradient algorithm acts like a Q-Learning [17] reinforcement learning implementation, but for continuous action spaces [9]. Like Q-Learning, the objective of the DDPG agent's policy is to select an action a which produces the maximum value for the Q-function at a given state s . For discrete action spaces, where there are a finite number of actions possible at a given state, it is easy to select the action which maximizes the Q function. However, for continuous action spaces, where an infinite number of actions are possible, evaluating the Q function for every action in the environment is not possible.

To address this challenge with conventional Q learning, the deep deterministic policy gradient algorithm used as actor-critic method to determining the optimal action to take in a given state [8], [9]. An actor neural network takes the current state as an input feature vector, and outputs an action for that provided state. The critic network is designed to input a state-action pair and predict the Q value of the given state-action pair. The critic network is continuously trained on random samples of state-action-reward tuples stored in a replay buffer which saves a history of the agent's interactions with the environment.

Based on the critics predicted Q value for an actor's action a at a given state s , it is possible to define a loss function for the actor which can

Full details on the implementation and equations behind the deep deterministic policy gradient algorithm can be found in **Appendix A**.

2.3.1. Implementation

To implement the deep deterministic policy gradient (DDPG) algorithm, the Stable Baselines 3 [18] library was used. Stable Baselines 3 was selected as it provides a collection of reliable implementation of common reinforcement learning algorithms (including DDPG) implemented in PyTorch [19]. The Stable Baselines 3 implementations are designed to be easy to use, provide direct integration with Gymnasium environments, and are configured with default hyperparameters which produce good results in a wide range of environments.

2.3.2. Hyperparameter Tuning

The default hyperparameters in the Stable Baselines 3 implementation of the DDPG algorithm were designed to be stable across a large range of different environments. As a

result, they largely worked well and provided good performance for the simulation environment developed. One key hyperparameter we changed was an increase in the gaussian noise added to the action space used to increase the variance and prevent the model from converging to suboptimal local minima during training. Recall, the action space was the turning rate of the front wheel, with the rate belonging to $[-1, 1]$ rad/s. Increasing the noise on the action output to a gaussian distribution with a mean of 0 and standard deviation of 0.2 was found to significantly decrease the time the DDPG model converged to a well performing model which followed the path closely, as adding the noise helped the DDPG model escape suboptimal local minima in the loss function.

The final hyperparameters used for the DDPG model are shown below, with all parameters besides action noise being DDPG defaults.

Table 3: Final Hyperparameters used for Decision Transformer Model

Hyperparameter	Value
Learning Rate	1×10^{-3}
Batch Size (Critic Training)	256
Replay Buffer Size (Time Steps)	50,000
Action Noise	$N(0, 0.2)$
Discount Factor	0.99
Network Layers (Actor & Critic)	Input $\rightarrow$ 2 Hidden Layers $\rightarrow$ Output
Hidden Layer 1 Neurons	400
Hidden Layer 2 Neurons	300

2.4. Decision Transformer Algorithm

Decision transformers take a different approach to reinforcement learning compared to traditional methods. Rather than conventional approaches which try to learn a policy for given states, or assign value functions to states, decision transformers function by treating the problem faced in reinforcement learning as a sequence modelling problem [10].

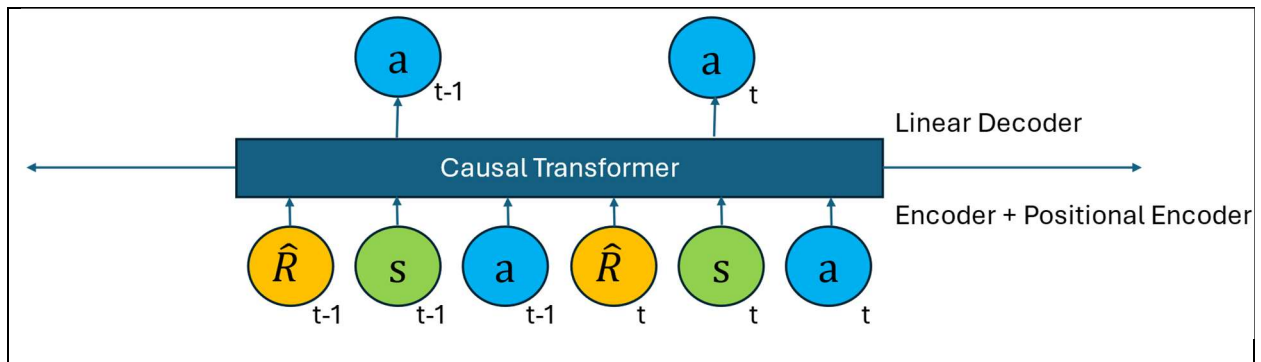


Figure 4: Decision Transformer Architecture. Based on figure by Chen et Al. [10]

Decision transformers utilize the transformer neural network architecture [20] for modelling the sequences. Transformer neural networks excel at identifying relationships in long-term sequences of data, and have seen large scale adoption in natural language processing, image models, and speech models [21]. The ability of transformer models to handle large time-dependent sequences of data and apply self-attention made transformer models compelling for the domain of reinforcement learning.

Rather than modelling rewards directly, the decision transformer models “return-to-go’s”, given by the equation [10]:

$$\hat{R} = \sum_{t'=t}^T r_{t'} \quad (15)$$

The reason this was selected for the decision transformer architecture is due to the fact we want to maximize the future return of a given action, not just the immediate reward. A sequence of length n timesteps of return-to-go’s, states, and actions is embedded and used to train the decision transformer.

The loss function for the decision transformer is based off the predicted action versus actual action for a given state, action, return-to-go pair in the training dataset. Since the objective of the decision transformer model is to output an action which can produce a desired return, the model must learn which actions produce a given future return [10].

$$L = mean \left((a_{preds, t+1} - a_{training, t+1})^2 \right) \quad (16)$$

For the inference loop, a target return is defined, typically, the return expected for an expert agent. In the case of our model, the target return was $1 * n$, where 1 is the reward for an agent perfectly following the path, and n was the episode subset length used for training the transformer.

The key concept which enables the decision transformer to function is that it was trained on previous time step state, action, return-to-go pairs to predict which action comes next. When operating at inference time, previous state, action, and return-to-go’s are fed to the transformer model, alongside the state, action, and the *target return* at the current time step t . By feeding the target return at the current time step, the decision transformer is therefore able to predict the action at time step $t + 1$ which should produce the desired return.

It should be noted that decision transformers are an offline reinforcement learning model. Whereas the DDPG model trains by directly interacting with the environment, decision transformer models are trained on recorded offline data of an agent interacting with the environment, rather than interacting with the environment itself. However, like other offline reinforcement learning algorithms such as the temporal difference algorithm, decision transformers are designed to perform better than the recorded data used to train them [10].

Zheng et Al. [22] proposed an architecture for an online decision transformer, where an offline decision transformer is trained, followed by online training of a decision transformer interacting with the environment, however, this was not implemented for the scope of this project.

2.4.1. Implementation

The Hugging Face Datasets library [23] were used to record 100,000 timesteps of the deep determinist policy gradient model interacting with the environment. This was used to generate the offline training set used to train the decision transformer model.

The Hugging Face transformer library [24] was selected for implementing the model itself as it offers pre-defined architectures to implement the decision transformers using PyTorch. The code used to implement the decision transformer was developed based on example code provided by Hugging Face [25]. Following the example code, a collator was used to pre-process the offline data into batches used for training the decision transformer. This collator normalized the data and scaled rewards to match Chen et Al.'s [10] initial implementation of the algorithm.

2.4.2. Initial Hyperparameter Trials

The initial attempts at the decision transformer, following the hyperparameters proposed in the sample code [25], showed characteristics of extreme overfitting. For the initial trials of the decision transformer, a batch size of 64 was used, with 1000 episodes of expert data generated by the DDPG model as the training dataset, a subset of 20 timesteps used for the training batches, and 64 epochs of training.

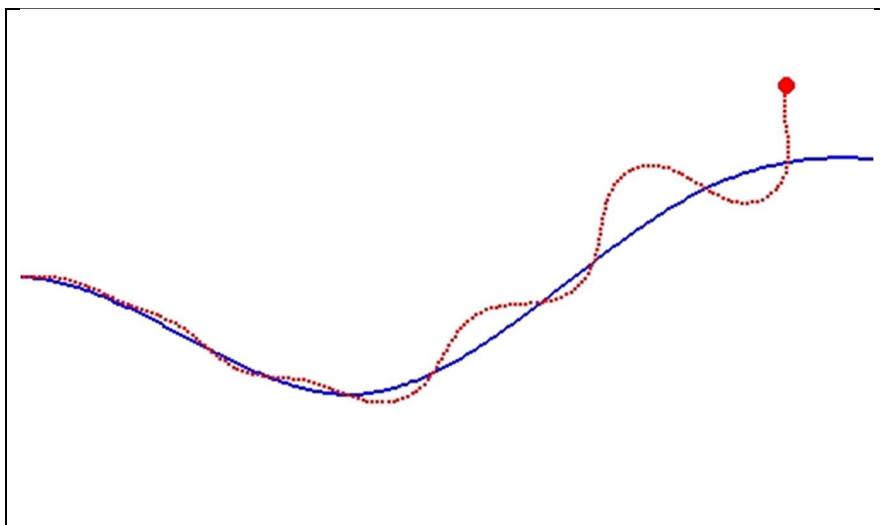


Figure 5: Unstable oscillatory behavior of agent (red) following desired path (blue) with initial hyperparameters

With these initial attempts, the decision transformer failed to stably converge to follow the path (Figure 5). The agent would oscillate about the desired path with increasing

amplitude and ultimately diverge from the path. This is suspected to be due to overfitting the training data and, as a result, the decision transformer model did not generalize well to new data.

2.4.3. Hyperparameter Tuning and Increasing Train Dataset Variance

Initial attempts at training the decision transformer failed to produce a stable controller which was capable of converging and following the path as discussed above. Two techniques were used to address this issue and attempt to develop a stable decision transformer model which was capable of stably following the path without diverging.

First, it was suspected that there was a lack of diversity of the states experiences in the offline training set used to train the decision transformer model. Recall, the state was defined as the cross-track error and heading error. Since the DDPG model performed well, the cross-track and heading errors were always low. It was suspected that since the offline data used to train the decision transformer did not contain a significant amount of states with higher errors, the decision transformer struggled to extrapolate actions which would produce high returns in such states. To address this issue, the diversity of states in the recorded training data from the DDPG model was artificially increased by adding step discontinuities to the generated paths. This added states with high cross-track error, and forced states with higher heading error to converge back to the path.

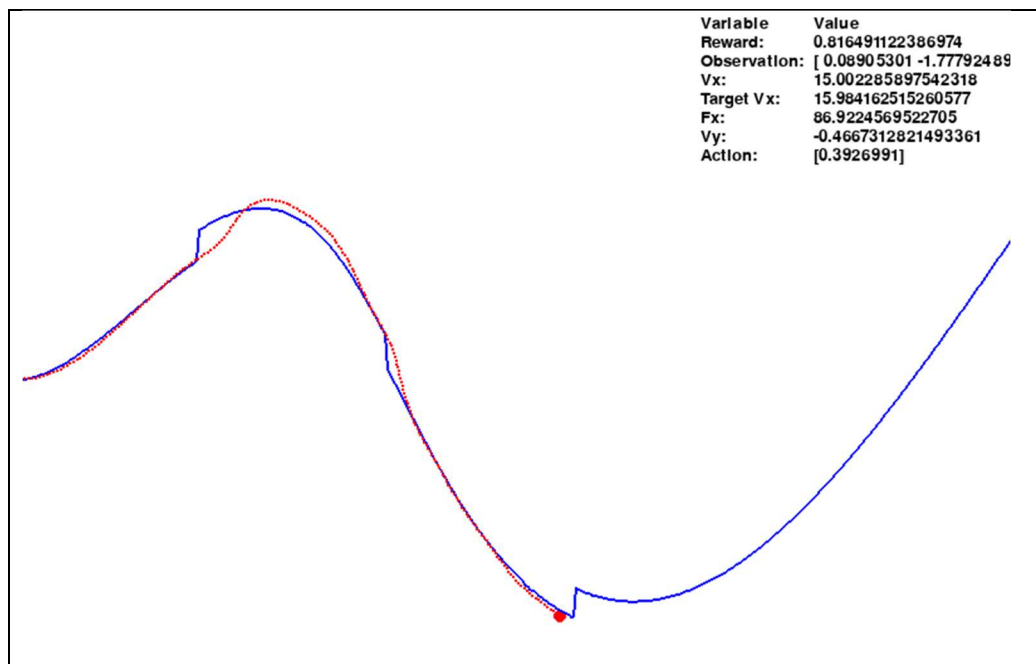


Figure 6: Step discontinuities on the desired path (blue) to increase the diversity of states recorded from the DDPG agent (red) to be used as offline training data for the decision transformer model

The second step that was taken to improve the performance of the decision transformer model was to tune the hyperparameters. The hyperparameters were adjusted to tuned to train significantly less than the example code provided by Hugging Face [25] so as to not

overfit. Specifically, the number of epochs was reduced from 64 to 2, and the size of the batch was reduced from 64 to 8. These values were found experimentally through trial and error to empirically maximize performance without showing behavior of overfitting.

Table 4: Final Hyperparameters used for Decision Transformer Model

Hyperparameter	Value
Epochs	2
Batch Size	8
Episode Length	20
Learning Rate	1×10^{-4}
Network Size	3 Layers
Layer Sizes	128 Layers, Fully Connected
Target Return-To-Go	20 (Reward of 1 x Episode Length of 20)

With these modifications, the decision transformer was able to stably converge and follow the path produced. Results and discussion of the performance of the decision transformer with these parameters are discussed in the sections below.

3. Results

The performance of the Deep Deterministic Policy Gradient and Decision Transformer models was evaluated using mean absolute cross-track error. Additionally, a Proportional Integral Derivative (PID) controller was implemented to test the performance of learning-based driving methods compared to conventional controls methods. All 3 controllers were able to follow a randomly generated path, but the mean absolute cross-track error of the three controllers was significantly different. A sample path taken by the DDPG, Decision Transformer, and PID controllers plotted over the reference path are shown below in Figure 1Figure 7, Figure 8, and Figure 9 respectively.

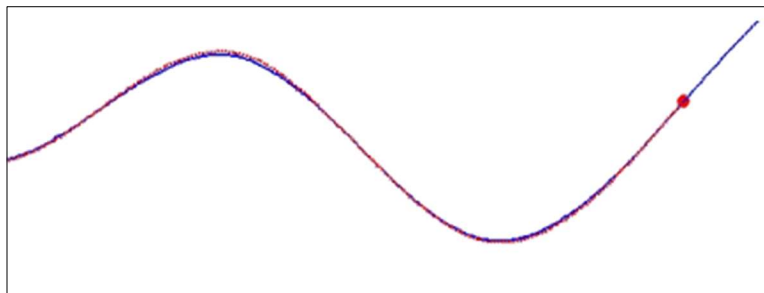


Figure 7: DDPG Path Tracking

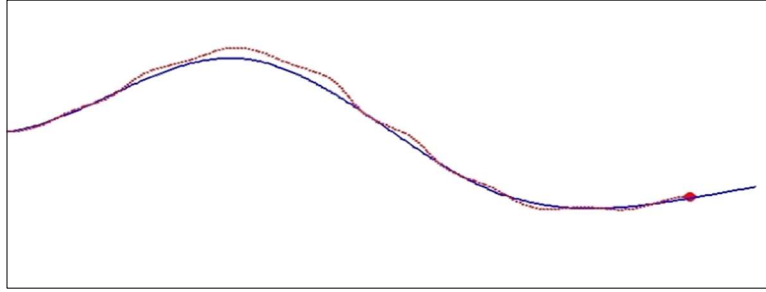


Figure 8: Decision Transformer Path Tracking

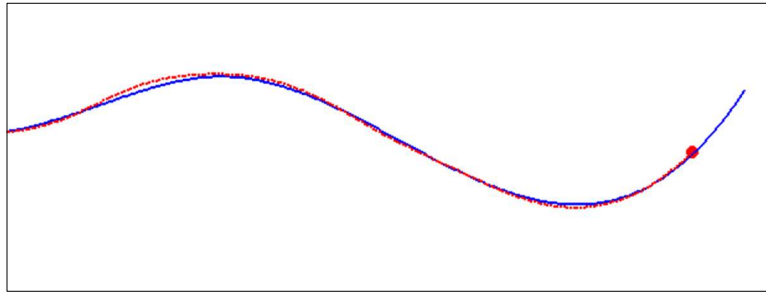


Figure 9: PID Controller Path Tracking

The cross-track error was recorded for the three controllers over 50 episodes, where each episode had a new randomly generated path. The recorded cross-track error was then used to find the mean absolute cross-track error for each controller. The performance of the three controllers is summarized in Figure 10 and Table 5.

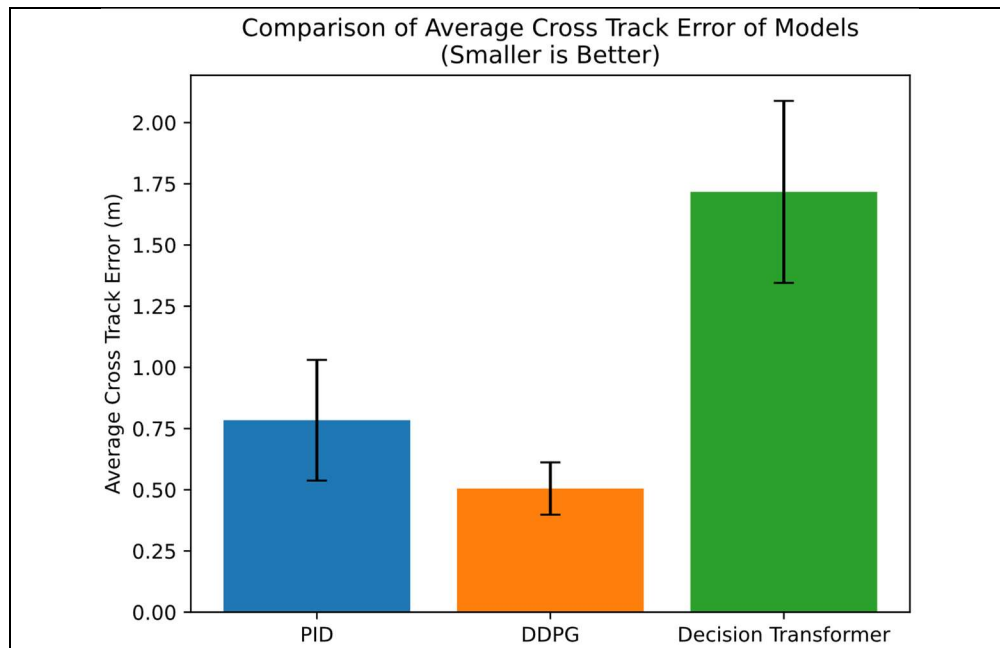


Figure 10: Comparison of average absolute cross-track error for the 3 models tested

Table 5: Mean Absolute Cross Track Errors for Models Tested

Model	Cross-Track Error (m)	Error Standard Deviation (m)
PID Controller	0.784	0.207
DDPG Model	0.505	0.107
Decision Transformer	1.717	0.372

As seen in Figure 10 and Table 5, the DDPG model performed the best, with a mean absolute cross-track error of 0.505 meters and a standard deviation of 0.107, followed closely by the PID controller with mean absolute cross-track error of 0.784 meters and a standard deviation of 0.207. The Decision Transformer performed the worst by a significant amount, with a mean absolute cross-track error of 1.717 meters and standard deviation of 0.372.

4. Discussion

The DDPG model clearly performed the best out of all three controllers tested. In fact, it performed well enough to be able to keep a Tesla Model S steered in the lanes of a Canadian highway. The Geometric Design Guide for Canadian Roads suggests a minimum lane width of 3.7 meters [26]. The Tesla Model S is 1.9 meters [27], leaving 1.8 meters of space in the lanes. If the determined path was in the middle of the lane, then at one standard deviation of error (0.612 meters) there would still be 0.3 meters from the edge of the vehicle to the edge of the lane. It is also important to note that the paths generated included much sharper turns than what would be experienced in a highway. All of the models performed significantly better on straighter paths, and most of the error observed happened in sharp turns.

It was also interesting to see that once the environment and vehicle model were set up, the implementation of the DDPG model was the simplest and took even less time than implementing a PID controller. The only development required for the DDPG model was to define the reward function and pass information into the observation space, compared to the PID controller which required manually tuning parameters to achieve acceptable levels of performance. The Decision Transformer performed the worst and required the most development and iteration to tune hyperparameters and select data that was useful for training the model.

In theory, the decision transformer should be able to perform better than the data it was trained on, because the decision transformer has access to the error in the training data and is training with the goal of minimizing that error. One of the reasons the performance of the Decision Transformer was so bad was due to the oscillations in the path. The root cause of these oscillations is still under investigation, however, there are two primary theories. The first possibility is that the oscillations are caused by the sinusoidal nature of the paths that are generated. The piecewise discontinuities added to the DDPG paths for training were also

added at fixed points, which could have caused the decision transformer to learn these points and predict that a discontinuity was coming, but then oscillate when the step did not happen in testing. The amount and diversity of the training data available could still be an issue as well, and generating a larger and more diverse data set while modifying the training hyperparameters could improve performance.

5. Conclusion

This project successfully demonstrated the implementation of learning-based controls including Deep Deterministic Policy Gradient and Decision Transformers for vehicle path following. The findings show that learning based controllers can outperform classical controllers, and also have additional benefits such as lower development time, and the ability to learn simple logic like slowing down in a turn when trained with proper reward functions.

Despite the promising results of the DDPG model, the challenges encountered with the Decision Transformer like the sensitivity to hyperparameters or need for more diverse data showed the need for more refinement in offline reinforcement learning approaches.

5.1. Future Work

This work provides a solid foundation for further exploration of reinforcement learning applications for self driving vehicles and path following.

As path following is only one aspect of self-driving vehicles, future works in this project will include the implementation of path planning or obstacle avoidance techniques using computer vision. One project of interest in particular is to modify the DDPG model to include a convolutional neural network (CNN) capable of processing images as the input. This would enable an end-to-end approach to self-driving, combining perception planning and control using only a series of images as input. The image input could be a birds eye view occupancy grid, and the model could learn to navigate through the environment on that data alone.

Another area of interest is to train models with knowledge of future points on the path and implement a learning based controller to produce results similar to a model predictive controller. Theoretically, if the model has information about the future path, it could be proactive and learn to minimize the total error along the path, instead of only minimizing the immediate error at the current time. This could reduce the long term error, and could also be very helpful in training a model that can slow down ahead of time in, or perform smoother actions in general because it has more time to prepare.

6. References

- [1] “Mechatronics Vehicle Systems Lab.” [Online]. Available: <https://uwaterloo.ca/mvs-lab/>
- [2] “(PDF) Perception, Planning and Control for Self-Driving System Based on On-board Sensors,” *ResearchGate*, Oct. 2024, doi: 10.1177/1687814020956494.
- [3] Z. Wang *et al.*, “Driverless simulation of path tracking based on PID control,” *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 892, no. 1, p. 012050, Jul. 2020, doi: 10.1088/1757-899X/892/1/012050.
- [4] Z. Wang, K. Sun, S. Ma, L. Sun, W. Gao, and Z. Dong, “Improved Linear Quadratic Regulator Lateral Path Tracking Approach Based on a Real-Time Updated Algorithm with Fuzzy Control and Cosine Similarity for Autonomous Vehicles,” *Electronics*, vol. 11, no. 22, Art. no. 22, Jan. 2022, doi: 10.3390/electronics11223703.
- [5] P. Stano *et al.*, “Model predictive path tracking control for automated road vehicles: A review,” *Annual Reviews in Control*, vol. 55, pp. 194–236, Jan. 2023, doi: 10.1016/j.arcontrol.2022.11.001.
- [6] Y. Cao *et al.*, “Path following for Autonomous Ground Vehicle Using DDPG Algorithm: A Reinforcement Learning Approach,” *Applied Sciences*, vol. 13, no. 11, p. 6847, Jun. 2023, doi: 10.3390/app13116847.
- [7] G. Hess and W. Ljungbergh, “Deep Deterministic Path Following,” Apr. 13, 2021, *arXiv*: arXiv:2104.06014. doi: 10.48550/arXiv.2104.06014.
- [8] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic Policy Gradient Algorithms”.
- [9] “Deep Deterministic Policy Gradient — Spinning Up documentation.” Accessed: Oct. 08, 2024. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ddpg.html#quick-facts>
- [10] L. Chen *et al.*, “Decision Transformer: Reinforcement Learning via Sequence Modeling,” Jun. 24, 2021, *arXiv*: arXiv:2106.01345. Accessed: Oct. 13, 2024. [Online]. Available: <http://arxiv.org/abs/2106.01345>
- [11] B. Varma, N. Swamy, and S. Mukherjee, “Trajectory Tracking of Autonomous Vehicles using Different Control Techniques(PID vs LQR vs MPC),” in *2020 International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*, Oct. 2020, pp. 84–89. doi: 10.1109/ICSTCEE49637.2020.9276986.
- [12] H. B. Pacejka, *Tire and Vehicle Dynamics*. Oxford: Butterworth-Heinemann, 2012. doi: 10.1016/B978-0-08-097016-5.00002-4.
- [13] S. Fuller, B. Greiner, J. Moore, R. Murray, R. van Paassen, and R. Yorke, “The Python Control Systems Library (python-control),” in *60th IEEE Conference on Decision and Control (CDC)*, IEEE, 2021, pp. 4875–4881.
- [14] G. Brockman *et al.*, “OpenAI Gym,” Jun. 05, 2016, *arXiv*: arXiv:1606.01540. Accessed: Oct. 08, 2024. [Online]. Available: <http://arxiv.org/abs/1606.01540>
- [15] “about - pygame wiki.” Accessed: Dec. 06, 2024. [Online]. Available: <https://www.pygame.org/wiki/about>
- [16] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

- [17] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Mach Learn*, vol. 8, no. 3, pp. 279–292, May 1992, doi: 10.1007/BF00992698.
- [18] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [19] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Dec. 03, 2019, *arXiv*: arXiv:1912.01703. Accessed: Oct. 08, 2024. [Online]. Available: <http://arxiv.org/abs/1912.01703>
- [20] A. Vaswani et al., “Attention Is All You Need,” Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
- [21] S. Islam et al., “A Comprehensive Survey on Applications of Transformers for Deep Learning Tasks,” Jun. 11, 2023, *arXiv*: arXiv:2306.07303. doi: 10.48550/arXiv.2306.07303.
- [22] Q. Zheng, A. Zhang, and A. Grover, “Online Decision Transformer,” Jul. 13, 2022, *arXiv*: arXiv:2202.05607. doi: 10.48550/arXiv.2202.05607.
- [23] Q. Lhoest et al., “Datasets: A Community Library for Natural Language Processing,” Sep. 07, 2021, *arXiv*: arXiv:2109.02846. doi: 10.48550/arXiv.2109.02846.
- [24] T. Wolf et al., “HuggingFace’s Transformers: State-of-the-art Natural Language Processing,” Jul. 14, 2020, *arXiv*: arXiv:1910.03771. doi: 10.48550/arXiv.1910.03771.
- [25] “Train your first Decision Transformer.” Accessed: Dec. 07, 2024. [Online]. Available: <https://huggingface.co/blog/train-decision-transformers>
- [26] Transportation Association of Canada (TAC), *Geometric Design Guide for Canadian Roads*. Ottawa: TAC, 2020.
- [27] “Model S Owner’s Manual,” Tesla. Accessed: Dec. 10, 2024. [Online]. Available: https://www.tesla.com/ownersmanual/models/en_us/GUID-91E5877F-3CD2-4B3B-B2B8-B5DB4A6C0A05.html

Appendix A: Deep Deterministic Policy Gradient Algorithm

The deep deterministic policy gradient was developed in the paper “Deterministic Policy Gradient Algorithms” by Silvers et al. [8] to provide a methodology to apply Q-Learning approaches to continuous action spaces. It functions by utilizing a critic neural network which is trained to predict the Q value for a given state action pair. This critic network is trained on a replay buffer of state-action-reward history. An actor neural network is trained to predict an action for a given state. A loss function for the actor network is developed using the Q value predicted from the critic network.

The critic network is trained to predict a Q value using a mean squared error bellman loss function which is used to approximate how close the predicted Q value for a state action pair is from the predicted Q value given by the Bellman equation. This loss function is given as [9]:

$$L(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(Q_{\phi}(s, a) - \left(r + \gamma(1 - d) \max_{a'} Q_{\phi}(s', a') \right) \right)^2 \right] \quad (\text{A1})$$

Where

- ϕ are the neural network parameters for the critic neural network
- $\mathcal{D}$ is the replay buffer set containing a subset of the recent state-action-reward-done data tuples
- s is the current state, and s' is the future state
- a is the action
- r is the reward
- γ is the discount factor
- d is a binary value of whether the agent is at a terminal state

For this loss function, some issues arise when selecting an action which maximizes the Q value for the action which maximizes the Q value in state s' (which is done in a similar manner to standard Q learning). Two problems arise when doing this. The first problem is that predicting Q value for the future state action pair relies on the same critic network parameters ϕ that we are trying to optimize with the loss function. In practice, if this is attempted, it is challenging to attain stability when training the critic neural network. To address this challenge, target neural networks are used for the purpose of approximating the

Q function when generating the loss to train the critic neural network. This target network is simply a time-delayed versions of the critic network [8], [9].

The second challenge faced is predicting the action which maximizes the Q value for the future state s' . Fortunately, for this problem, the actor neural network is designed to select the action which maximizes the Q value for a given state. Similarly to above, a target network is used with a time delay to assist with stability of training the critic network [9].

With a loss function for the critic network defined, all that remains to have a function deep deterministic policy gradient algorithm is a method to train the actor neural network. To update the actor neural network, gradient ascent is conducted with respect to the actor neural networks parameters to solve, given by the equation [9]:

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q_{\phi}(s, \mu_{\theta}(s))] \quad (\text{A2})$$

Where

- μ_{θ} is the actor neural network policy which produces an action a given a state s
- θ are the neural network parameters of the actor neural network
- Q_{ϕ} is the critic network with policies that have already been trained in the previous step

In short, the policy update step performs gradient ascent on the parameters of the actor neural network to maximize the critic network's prediction of the Q value for the action produced by the actor network in a given state s .